

SMART FILE ORGANIZER USING PYTHON

Name : LAKSHMI . B

1. Project Title

Smart File Organizer Using Python

2. Objective

The objective of this project is to automate file management tasks using Python. The system automatically sorts files into folders based on file type, records operations in a log file, accepts user input, and handles errors using exception handling.

3. Problem Statement

Managing large numbers of files manually can be time-consuming and error-prone. This project provides an automated solution for organizing files into categorized folders.

4. Requirements

- Python Programming Language
- OS Module
- shutil Module
- logging Module
- VS Code

5. Modules Used

OS Module

Used for:

- Creating folders
- Accessing directories
- Checking file paths

shutil Module

Used for:

- Moving files between folders

logging Module

Used for:

- Recording operations in a log file

6. Features

- Organizes files automatically
- Supports multiple file formats
- User input support
- Generates operation logs
- Uses exception handling
- Easy to use

7. Project Structure

Smart-File-Organizer/

|— file_organizer.py

|— README.md

|— operations.log

|— SampleFiles/

|— Screenshots/

|— Project_Report.pdf

8. Source Code

```
import os
import shutil
import logging
```

```

# Configure logging
logging.basicConfig(
    filename="operations.log",
    level=logging.INFO,
    format="%(asctime)s - %(message)s"
)

# File categories
FILE_TYPES = {
    "Images": [".jpg", ".jpeg", ".png", ".gif"],
    "Documents": [".pdf", ".docx", ".txt"],
    "Audio": [".mp3", ".wav"],
    "Videos": [".mp4", ".avi", ".mkv"],
    "Archives": [".zip", ".rar"]
}

def get_category(extension):
    for category, extensions in FILE_TYPES.items():
        if extension.lower() in extensions:
            return category
    return "Others"

def organize_files(folder_path):
    try:
        files = os.listdir(folder_path)

        for file in files:

            file_path = os.path.join(folder_path, file)

            if os.path.isfile(file_path):

                _, extension = os.path.splitext(file)

                category = get_category(extension)

                category_folder = os.path.join(folder_path, category)

                if not os.path.exists(category_folder):
                    os.makedirs(category_folder)

                destination = os.path.join(category_folder, file)

                shutil.move(file_path, destination)

                print(f"Moved: {file} → {category}")
    
```

```

        logging.info(
            f"{file} moved to {category}"
        )

    print("\nFile Organization Completed Successfully!")

except FileNotFoundError:
    print("Folder not found!")

except PermissionError:
    print("Permission denied!")

except Exception as e:
    print("Unexpected Error:", e)

if __name__ == "__main__":
    folder = input("Enter folder path: ")

    organize_files(folder)

```

9. Sample Input

Enter folder path: SampleFiles

10. Sample Output

Moved: notes.txt → Documents

Moved: photo.jpg → Images

Moved: report.pdf → Documents

Moved: song.mp3 → Audio

File Organization Completed Successfully!

```

● PS C:\Users\admin\Downloads\Smart-File-Organizer> python file_organizer.py
Enter folder path: Screenshots
Moved: execution.png → Images
Moved: input_folder.png → Images
Moved: log_file.png → Images
Moved: output_folder.png → Images

File Organization Completed Successfully!

```

11. Exception Handling

The program handles:

- Invalid folder paths
- Permission errors
- Unexpected runtime errors

This prevents the program from crashing.

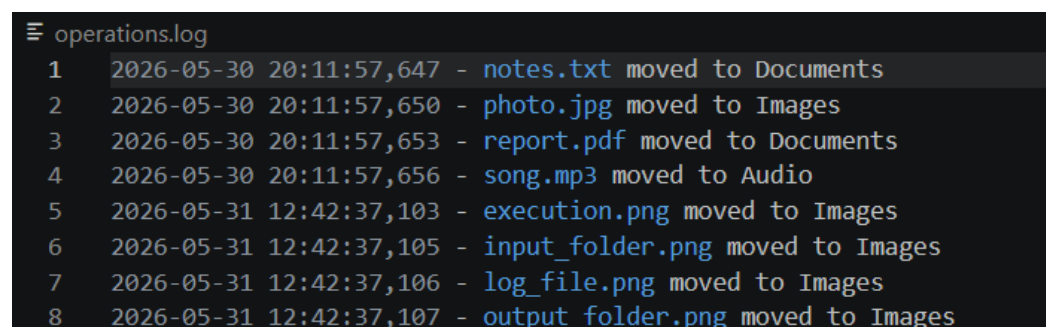
12. Advantages

- Saves time
- Reduces manual effort
- Improves file organization
- Generates logs for tracking
- User friendly

13. Applications

- Personal file management
- Office document organization
- Educational projects
- Data organization

14. Screenshots



```
≡ operations.log
1 2026-05-30 20:11:57,647 - notes.txt moved to Documents
2 2026-05-30 20:11:57,650 - photo.jpg moved to Images
3 2026-05-30 20:11:57,653 - report.pdf moved to Documents
4 2026-05-30 20:11:57,656 - song.mp3 moved to Audio
5 2026-05-31 12:42:37,103 - execution.png moved to Images
6 2026-05-31 12:42:37,105 - input_folder.png moved to Images
7 2026-05-31 12:42:37,106 - log_file.png moved to Images
8 2026-05-31 12:42:37,107 - output_folder.png moved to Images
```

15. Conclusion

The Smart File Organizer successfully automates file organization tasks. The project demonstrates practical usage of Python modules such as OS, shutil, logging, and exception handling. The system improves productivity by automatically sorting files and maintaining operation logs.

16. Github Repository Link :

<https://github.com/its-me-lakshmi-b/Smart-File-Organizer>